Комп’ютерний практикум №4

Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Виконали: Сирота Володимир ФБ-42
Сімутін Владислав ФБ-42

1. Мета роботи

Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

2. Постановка задачі та варіант завдання

Потрібно реалізувати власноруч, без готових бібліотечних реалізацій тестів простоти та RSA:

1. Функцію пошуку випадкового простого числа заданої довжини з використанням тесту Міллера—Рабіна та попередніх пробних ділень.
2. Генерацію двох пар простих чисел p, q та p_1, q_1 довжиною щонайменше 256 біт, причому $pq \le p_1q_1$ (пара p, q — абонент A, пара p_1, q_1 — абонент B).
3. Функцію GenerateKeyPair(), що повертає відкритий ключ (n, e) та секретний ключ (d, p, q), і побудову схем RSA для абонентів A та B.
4. Процедури Encrypt(), Decrypt(), Sign(), Verify(), кожна з яких приймає лише необхідні їй ключові дані.
5. Протокол конфіденційного розсилання ключів із підтвердженням справжності відправника: процедури SendKey() та ReceiveKey().
6. Перевірку взаємодії реалізації з тестовим середовищем http://asymcryptwebservice.appspot.com/?section=rsa.

Варіант завдання: загальний для всіх бригад; відкритий показник $e = 2^{16} + 1 = 65537$ (у тестовому середовищі відображається як 10001 у шістнадцятковій системі).

3. Хід роботи

3.1. Структура програми

Програма складається з п'яти модулів мовою Python 3.11:

- **part1.py** — власна реалізація піднесення до степеня за модулем (схема Горнера, mod_pow), тест Міллера—Рабіна з пробними діленнями (miller_rabin) та генерація простого числа заданої довжини (generate_prime);
- **part2.py** — генерація двох пар простих чисел із виконанням умови $pq \le p_1q_1$ (generate_two_pairs);
- **part3.py** — розширений алгоритм Евкліда, пошук оберненого елемента, GenerateKeyPair() та збереження ключів у файл keys.txt;
- **part4.py** — Encrypt(), Decrypt(), Sign(), Verify() та завантаження ключів із файлу;
- **part5.py** — SendKey() та ReceiveKey(), протокол передачі ключа A → B.

3.2. Піднесення до степеня за модулем

Функція mod_pow(base, exponent, modulus) реалізує швидке піднесення до степеня за модулем: показник розглядається в двійковому вигляді, на кожному кроці виконується

піднесення основи до квадрата за модулем, а при непарному залишку показника — множення результату на поточну основу. Усі проміжні обчислення виконуються за модулем згідно з властивістю конгруенцій, тому розмір чисел не перевищує розміру модуля.

3.3. Генерація простого числа

Функція generate_prime(bits) формує випадкове число заданої довжини (random.getrandbits), встановлює старший біт (щоб довжина була рівно bits) і молодший біт (щоб число було непарним). Далі кандидат перевіряється:

1. **Пробними діленнями** на малі прості числа 2, 3, 5, 7, …, 47 — швидко відсікає значну частину складених чисел;
2. **Тестом Міллера—Рабіна** з 40 випадковими основами. Число $p - 1$ розкладається у вигляді $d \cdot 2^s$ з непарним d; для кожної основи a обчислюється $x_0 = a^d \bmod p$, і число проходить раунд, якщо $x_0 = \pm 1$ або один з наступних квадратів $x_r = x_{r-1}^2 \bmod p$ дорівнює -1. Якщо жоден раунд не виявив складеності, число вважається простим із імовірністю помилки не більше 4^{-40}.

Якщо кандидат не пройшов перевірку, генерується наступний випадковий кандидат.

3.4. Генерація ключів RSA

GenerateKeyPair(p, q) перевіряє $p \neq q$, обчислює $n = pq$, $\varphi(n) = (p - 1)(q - 1)$, приймає $e = 65537$, перевіряє $\gcd(e, \varphi(n)) = 1$ та знаходить $d = e^{-1} \bmod \varphi(n)$ за розширеним алгоритмом Евкліда. Відкритий ключ — (n, e), секретний — (d, p, q). Якщо для згенерованих чисел e не підходить ($\gcd \neq 1$), пара генерується заново.

3.5. Операції RSA

Кожна операція реалізована окремою процедурою, що отримує лише потрібні дані:

- Encrypt(M, (n, e)) повертає $C = M^e \bmod n$;
- Decrypt(C, (d, p, q)) повертає $M = C^d \bmod n$, де $n = pq$;
- Sign(M, (d, p, q)) повертає $S = M^d \bmod n$;
- Verify(M, S, (n, e)) повертає true, якщо $S^e \bmod n = M$.

3.6. Протокол розсилання ключів

SendKey(k, секретний ключ A, відкритий ключ B) перевіряє $0 < k < n$ та умову $n \leq n_1$, підписує ключ: $S = k^d \bmod n$, шифрує обидва значення відкритим ключем B: $k_1 = k^{e1} \bmod n_1$, $S_1 = S^{e1} \bmod n_1$ і повертає пару (k_1, S_1).

ReceiveKey(k_1, S_1, секретний ключ B, відкритий ключ A) перевіряє $n \leq n_1$, розшифровує $k = k_1^{d1} \bmod n_1$ та $S = S_1^{d1} \bmod n_1$, перевіряє $0 < k < n$, а потім перевіряє підпис: $S^e \bmod n = k$. Якщо підпис неправильний, ключ відхиляється.

3.7. Труднощі та шляхи їх розв'язання

- **Умова $n \le n_1$.** Протокол коректний лише тоді, коли модуль відправника не перевищує модуль отримувача: інакше підпис S (число < n) може не вміститися в діапазон $[0, n_1)$. У generate_two_pairs пари впорядковуються так, щоб $pq \le p_1q_1$, а SendKey і ReceiveKey перевіряють цю умову явно.
- **Тестування із сервером.** Сервер приймає і видає числа в шістнадцятковому вигляді, тому в консолі використовувався формат format(x, "X") та int(..., 16). Для напрямку A → сервер потрібен серверний модуль не менший за наш (використано серверний ключ довжиною 1024 біти), а для напрямку сервер → A — не більший за наш (використано серверний ключ довжиною 256 біт).
- **Режим Bytes.** У тестовому середовищі повідомлення вводиться як байти (hex): значення 3039 на сайті відповідає числу 12345 у нашій програмі (0x3039 = 12345).
- **Власні реалізації.** Піднесення до степеня за модулем, тест Міллера—Рабіна та обернений елемент реалізовані самостійно, без використання pow(a, b, n) та бібліотек для простих чисел.

4. Завдання 1. Генерація випадкового простого числа

Реалізовано функцію пошуку випадкового простого числа заданої довжини: випадковий кандидат (вбудований генератор random) перевіряється пробними діленнями та власною реалізацією тесту Міллера—Рабіна з 40 раундами. Також у коді реалізовано піднесення до степеня за модулем за схемою Горнера. Для одного з простих чисел наведено проміжні результати тесту для однієї з основ.

(part1.py)

```
import random

SMALL_PRIMES = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47)


def mod_pow(base, exponent, modulus):
    result = 1

    while exponent > 0:
        if exponent % 2 == 1:
            result = result * base % modulus

        base = base * base % modulus
        exponent //= 2

    return result


def miller_rabin(n, rounds=40):
    if rounds < 1:
```

```
    raise ValueError("Кількість раундів має бути додатною")

if n < 2:
    return False, None

for prime in SMALL_PRIMES:
    if n == prime:
        return True, None
    if n % prime == 0:
        return False, None

d = n - 1
s = 0

while d % 2 == 0:
    d //= 2
    s += 1

example = None

for i in range(rounds):
    a = random.randrange(2, n - 1)
    x = mod_pow(a, d, n)

    values = [x] if i == 0 else None
    passed = x == 1 or x == n - 1

    if not passed:
        for r in range(1, s):
            x = x * x % n

            if i == 0:
                values.append(x)

            if x == n - 1:
                passed = True
                break

            if x == 1:
                break

    if not passed:
        return False, None

    if i == 0:
        example = (s, d, a, values)
```

```
    return True, example


def generate_prime(bits=256):
    if bits < 8:
        raise ValueError("У цій реалізації довжина має бути від 8 біт")

    while True:
        candidate = random.getrandbits(bits)
        candidate |= (1 << (bits - 1)) | 1

        is_prime, example = miller_rabin(candidate)

        if is_prime:
            return candidate, example


if __name__ == "__main__":
    p, example = generate_prime(256)
    s, d, a, values = example

    print("Згенероване ймовірно просте число:")
    print("p =", p)
    print("Бітова довжина:", p.bit_length())

    print("\nПроміжні результати Міллера—Рабіна:")
    print(f"p - 1 = 2^{s} * {d}")
    print("s =", s)
    print("d =", d)
    print("Використана основа a =", a)
    print("x0 = a^d mod p =", values[0])

    for r, value in enumerate(values[1:], start=1):
        print(f"x{r} = x{r - 1}^2 mod p = {value}")

    if values[0] == 1:
        print("Раунд пройдено: x0 = 1")
    elif values[0] == p - 1:
        print("Раунд пройдено: x0 = p - 1")
    else:
        print("Раунд пройдено: один із наступних квадратів дорівнює p - 1")
```

Результат виконання part1.py (демонстраційний запуск, random.seed(5))

```
Згенероване ймовірно просте число:
p =
```

1076339048114609708951608084611338343545752306579072199774182797446998349358 77
Бітова довжина: 256

Проміжні результати Міллера—Рабіна:
p - 1 = 2^2 *
269084762028652427237902021152834585886438076644768049943545699361749587339 69
s = 2
d =
269084762028652427237902021152834585886438076644768049943545699361749587339 69
Використана основа a =
454355253755435455270290324934792113779536048695341371035016249453493401088 85
x0 = a^d mod p =
329512345181567081672783227322525311180388310462085707009999465692325861455 39
x1 = x0^2 mod p =
107633904811460970895160808461133834354575230657907219977418279744699834935876
Раунд пройдено: один із наступних квадратів дорівнює p - 1
У цьому запуску $x_0 \neq \pm 1$, а $x_1 = p - 1$, тому раунд пройдено на кроці з квадратом.

Проміжні результати тесту Міллера—Рабіна

Для числа q_1 =
107629633939348689431558942657599647329816831989354223317896357258752587585037
(єдине з чотирьох чисел, для якого $s > 1$; для p, q, p_1 маємо $s = 1$) отримано розклад

$q_1 - 1 = 2^2 \cdot d$, $s = 2$,

d =
269074084848371723578897356643999118324542079973385558294740893146881468962 59.

Випадкова основа a =
136540528803234123796636924213288065470616118854899414382078733428318874956 71.

- $x_0 = a^d \bmod q_1$ =
  974042109275328958901178061743458553975396249559963763490656064516661530 87712 — не дорівнює 1 і не дорівнює $q_1 - 1$, тому переходимо до квадратів;
- $x_1 = x_0^2 \bmod q_1$ =
  107629633939348689431558942657599647329816831989354223317896357258752587 585036 = $q_1 - 1$.

Отже, число q_1 сильно псевдопросте за основою a, раунд пройдено (один із квадратів дорівнює $q_1 - 1$). Аналогічні перевірки пройдені для ще 39 випадкових основ.

5. Завдання 2. Дві пари простих чисел

За допомогою функції з завдання 1 згенеровано дві пари простих чисел p, q та p_1, q_1 довжиною по 256 біт. Пари впорядковано так, щоб $pq \leq p_1 q_1$: p, q належать абоненту A, p_1, q_1 — абоненту B.

(part2.py)

```
from part1 import generate_prime


def generate_pair(bits=256):
    p, _ = generate_prime(bits)
    q, _ = generate_prime(bits)

    while q == p:
        q, _ = generate_prime(bits)

    return p, q


def generate_two_pairs(bits=256):
    p, q = generate_pair(bits)
    p1, q1 = generate_pair(bits)

    if p * q > p1 * q1:
        p, q, p1, q1 = p1, q1, p, q

    return p, q, p1, q1


if __name__ == "__main__":
    p, q, p1, q1 = generate_two_pairs(256)

    print("Абонент A:")
    print("p =", p)
    print("q =", q)

    print("\nАбонент B:")
    print("p1 =", p1)
    print("q1 =", q1)

    print("\nБітові довжини:")
    for name, value in (("p", p), ("q", q), ("p1", p1), ("q1", q1)):
        print(f"{name}: {value.bit_length()} біт")

    print("\nДобутки:")
    print("p * q =", p * q)
    print("p1 * q1 =", p1 * q1)

    print("\nПеревірка p * q <= p1 * q1:")
    print(p * q <= p1 * q1)
```

Згенеровані прості числа

Всі чотири числа мають довжину рівно 256 біт і пройшли тест Міллера—Рабіна з 40 основами.

Абонент A:

p =
91878084699872161213101062270443460167219549525581992320586387990535258303263

q =
11458743213075669256571780600058144486736436456875947000432451622916641145001 1

Абонент B:

p_1 =
10433141765843526642577380647635350821339522338457743731101208569802579866932 3

q_1 =
10762963393934868943155894265759964732981683198935422331789635725875258758503 7

Кандидати, що не пройшли перевірку простоти

В ході генерації більшість випадкових кандидатів відкидається. Для ілюстрації наведено окремий запуск `generate_prime(256)` (генератор ініціалізовано `random.seed(2026)`): для знаходження простого числа знадобилось 59 кандидатів, з яких 58 було відхилено — 42 пробними діленнями на малі прості числа та 16 тестом Міллера—Рабіна. Приклади кандидатів, відхилених пробними діленнями:

11034414253576796616526802386667760220861767543294328934503120723615123129244 1

10659992855408611664723352479330591294641651658848609309510930527139887988134 9

11473889077260846377567586603464310398036870598588812331103164143323816677592 5

Знайдене у цьому запуску просте число:

72701006939494238243595020906218961312501607072826208109892606521075242172797

Примітка: програма під час основного запуску не зберігала список відхилених кандидатів, тому наведено приклад окремого запуску того самого алгоритму.

Результат виконання part2.py (демонстраційний запуск, `random.seed(5)`)

```
Абонент A:
p =
10763390481146097089516080846113383435457523065790721997741827974469983493587 7
q =
80417281833171205498779670605953459806547274059693823529727443771639834258057

Абонент B:
p1 =
```

```
871134190540174618818791285613841141299274135165052284573608391658108855588 69
q1 =
1143429085281503777930297836356360992185340388128717410995715522182124364410 69

Бітові довжини:
p: 256 біт
q: 256 біт
p1: 256 біт
q1: 256 біт

Добутки:
p * q =
865562605802797914131480679329028888419766571561155129724069391328727095496672
499189303448205135777207599133966331766337208294735356419833653422846561098 9
p1 * q1 =
996080170646795085857505378588007428422883181434291121391126601163736183430278
256075148163958862520413159443090308254450245269035546291783106737744879096 1

Перевірка p * q <= p1 * q1:
True
```

6. Завдання 3. Генерація ключових пар RSA

Функція `GenerateKeyPair(p, q)` повертає відкритий ключ (n, e) та секретний ключ (d, p, q). За її допомогою побудовано схеми RSA для абонентів A і B, ключі збережено у файлі keys.txt. Для однієї з пар наведено основні параметри та перевірку співвідношень, потрібних для коректної роботи RSA.

(part3.py)

```
from math import gcd
from pathlib import Path

from part2 import generate_two_pairs


def extended_gcd(a, b):
    old_r, r = a, b
    old_x, x = 1, 0

    while r != 0:
        quotient = old_r // r
        old_r, r = r, old_r - quotient * r
        old_x, x = x, old_x - quotient * x

    return old_r, old_x
```

```
def modular_inverse(a, modulus):
    divisor, x = extended_gcd(a, modulus)

    if divisor != 1:
        raise ValueError("Обернений елемент не існує")

    return x % modulus


def GenerateKeyPair(p, q):
    if p == q:
        raise ValueError("p і q повинні бути різними")

    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537

    if not 1 < e < phi or gcd(e, phi) != 1:
        raise ValueError("Для цих p і q показник e = 65537 не підходить")

    d = modular_inverse(e, phi)

    public_key = (n, e)
    private_key = (d, p, q)

    return public_key, private_key


def save_keys(filename, public_a, private_a, public_b, private_b):
    n, e = public_a
    d, p, q = private_a

    n1, e1 = public_b
    d1, p1, q1 = private_b

    values = (
        ("p", p),
        ("q", q),
        ("n", n),
        ("e", e),
        ("d", d),
        ("p1", p1),
        ("q1", q1),
        ("n1", n1),
        ("e1", e1),
```

```
        ("d1", d1),
    )

    with open(filename, "w", encoding="utf-8") as file:
        for name, value in values:
            file.write(f"{name} = {value}\n")


if __name__ == "__main__":
    while True:
        p, q, p1, q1 = generate_two_pairs(256)

        try:
            public_a, private_a = GenerateKeyPair(p, q)
            public_b, private_b = GenerateKeyPair(p1, q1)
            break
        except ValueError:
            continue

    n, e = public_a
    d = private_a[0]

    n1, e1 = public_b
    d1 = private_b[0]

    phi = (p - 1) * (q - 1)

    print("Основні параметри абонента A:")
    print("p =", p)
    print("q =", q)
    print("n =", n)
    print("phi(n) =", phi)
    print("e =", e)
    print("d =", d)

    print("\nВідкритий ключ A (n, e):")
    print(public_a)
    print("Секретний ключ A (d, p, q):")
    print(private_a)

    print("\nВідкритий ключ B (n1, e1):")
    print(public_b)
    print("Секретний ключ B (d1, p1, q1):")
    print(private_b)

    print("\nПеревірка співвідношень для A:")
```

```
    print("p != q:", p != q)
    print("n = p * q:", n == p * q)
    print("2 <= e <= phi(n) - 1:", 2 <= e <= phi - 1)
    print("gcd(e, phi(n)) = 1:", gcd(e, phi) == 1)
    print("e * d mod phi(n) = 1:", e * d % phi == 1)

    print("\nПеревірка порядку модулів:")
    print("n <= n1:", n <= n1)

    filename = Path(__file__).resolve().with_name("keys.txt")
    save_keys(filename, public_a, private_a, public_b, private_b)

    print("\nКлючі збережено в:", filename)
```

Параметри RSA для абонентів A та B

Абонент A:

$n = p \cdot q =$
105280737948505161563274206011052771286510154111409759568721971214092379007284709827257145026094225997393866144958971807512626211304618560334934234026858 93

$\varphi(n) = (p - 1)(q - 1) =$
105280737948505161563274206011052771286510154111409759568721971214092379007282645172088838737556437808711155895908625968371682796681369451292737217329326 20

$e = 65537$

$d =$
940389263891777721402033774155212807778967287414409557256727569602171868518137657465082095741003755570768017301767712217908958181627277157823144783136461 3

Відкритий ключ A: (n, e). Секретний ключ A: (d, p, q).

Абонент B:

$n_1 = p_1 \cdot q_1 =$
112291522909506875236510301311220366031745118948895863025929234859298388285561038458281963410569388313033045861298825875274250447746831352554029647057199 51

$\varphi(n_1) = (p_1 - 1)(q_1 - 1) =$
112291522909506875236510301311220366031745118948895863025929234859298388285558918847765985571010814985541706329743393754720511131140542268124461863194655 92

$e_1 = 65537$

$d_1 =$
912834456036572971771720221044078340587340427105509254096068772809860221600231466276085363578005123696495101457712291633198033328141893873170702080284188 9

Відкритий ключ B: (n_1, e_1). Секретний ключ B: (d_1, p_1, q_1).

Довжина модулів: n і n_1 — по 512 біт.

Перевірка співвідношень, необхідних для коректної роботи RSA (абонент A)

- $p \neq q$ — виконується;
- $n = p \cdot q$ — виконується;
- $2 \leq e \leq \varphi(n) - 1$ — виконується ($e = 65537$);
- $\gcd(e, \varphi(n)) = 1$ — виконується;
- $e \cdot d \bmod \varphi(n) = 1$ — виконується;
- $n \leq n_1$ — виконується ($n \approx 1{,}0528 \cdot 10^{154}$, $n_1 \approx 1{,}1229 \cdot 10^{154}$).

Ті самі співвідношення перевірені й для абонента B: $p_1 \neq q_1$, $n_1 = p_1 q_1$, $\gcd(e_1, \varphi(n_1)) = 1$, $e_1 d_1 \bmod \varphi(n_1) = 1$.

Результат виконання part3.py (на ключах із keys.txt)

Основні параметри абонента A:
p =
91878084699872161213101062270443460167219549525581992320586387990535258303263
q =
11458743213075669256571780600058144486736436456875947000432451622916641145001 1
n =
10528073794850516156327420601105277128651015411140975956872197121409237900728 4709827257145026094225997393866144958971807512626211304618560334934234026858 93
phi(n) =
10528073794850516156327420601105277128651015411140975956872197121409237900728 2645172088838737556437808711155895908625968371682796681369451292737217329326 20
e = 65537
d =
94038926389177772140203377415521280777896728741440955725672756960217186851813 7657465082095741003755570768017301767712217908958181627277157823144783136461 3

Відкритий ключ A (n, e):
(10528073794850516156327420601105277128651015411140975956872197121409237900728 4709827257145026094225997393866144958971807512626211304618560334934234026858 93, 65537)
Секретний ключ A (d, p, q):
(94038926389177772140203377415521280777896728741440955725672756960217186851813 7657465082095741003755570768017301767712217908958181627277157823144783136461 3, 91878084699872161213101062270443460167219549525581992320586387990535258303263, 11458743213075669256571780600058144486736436456875947000432451622916641145001 1)

Відкритий ключ B (n1, e1):
(11229152290950687523651030131122036603174511894889586302592923485929838828556 1038458281963410569388313033045861298825875274250447746831352554029647057199 51

```
, 65537)
Секретний ключ B (d1, p1, q1):
(91283445603657297177172022104407834058734042710550925409606877280986022160023
14662760853635780051236964951014577122916331980333281418938731707020802841889,
10433141765843526642577380647635350821339522338457743731101208569802579866932 3
,
10762963393934868943155894265759964732981683198935422331789635725875258758503 7
)

Перевірка співвідношень для A:
p != q: True
n = p * q: True
2 <= e <= phi(n) - 1: True
gcd(e, phi(n)) = 1: True
e * d mod phi(n) = 1: True

Перевірка порядку модулів:
n <= n1: True

Ключі збережено в: keys.txt
```

7. Завдання 4. Шифрування, розшифрування та цифровий підпис

Операції Encrypt(), Decrypt(), Sign() і Verify() реалізовано окремими процедурами, кожна з яких приймає лише потрібні їй ключові дані. Для демонстрації випадковим чином вибрано відкрите повідомлення M і виконано шифрування для A та B, розшифрування, підпис і перевірку підпису.

(part4.py)

```
import random
from pathlib import Path

from part1 import mod_pow


def load_keys(filename):
    values = {}

    with open(filename, "r", encoding="utf-8") as file:
        for line in file:
            if line.strip():
                name, value = line.split("=", 1)
                values[name.strip()] = int(value.strip())

    public_a = (values["n"], values["e"])
    private_a = (values["d"], values["p"], values["q"])
```

```
    public_b = (values["n1"], values["e1"])
    private_b = (values["d1"], values["p1"], values["q1"])

    return public_a, private_a, public_b, private_b


def Encrypt(message, public_key):
    n, e = public_key

    if not 0 <= message < n:
        raise ValueError("Повідомлення має задовольняти 0 <= M < n")

    return mod_pow(message, e, n)


def Decrypt(ciphertext, private_key):
    d, p, q = private_key
    n = p * q

    if not 0 <= ciphertext < n:
        raise ValueError("Криптограма має задовольняти 0 <= C < n")

    return mod_pow(ciphertext, d, n)


def Sign(message, private_key):
    d, p, q = private_key
    n = p * q

    if not 0 <= message < n:
        raise ValueError("Повідомлення має задовольняти 0 <= M < n")

    return mod_pow(message, d, n)


def Verify(message, signature, public_key):
    n, e = public_key

    if not 0 <= message < n or not 0 <= signature < n:
        return False

    return mod_pow(signature, e, n) == message


if __name__ == "__main__":
```

```
filename = Path(__file__).resolve().with_name("keys.txt")
public_a, private_a, public_b, private_b = load_keys(filename)

message = random.randrange(1, min(public_a[0], public_b[0]))

print("ВІДКРИТЕ ПОВІДОМЛЕННЯ")
print("M =", message)

print("\nШИФРУВАННЯ ДЛЯ АБОНЕНТА A")

ciphertext_a = Encrypt(message, public_a)
decrypted_a = Decrypt(ciphertext_a, private_a)

print("C_A = M^e mod n =", ciphertext_a)
print("M_A = C_A^d mod n =", decrypted_a)
print("M_A == M:", decrypted_a == message)

print("\nШИФРУВАННЯ ДЛЯ АБОНЕНТА B")

ciphertext_b = Encrypt(message, public_b)
decrypted_b = Decrypt(ciphertext_b, private_b)

print("C_B = M^e1 mod n1 =", ciphertext_b)
print("M_B = C_B^d1 mod n1 =", decrypted_b)
print("M_B == M:", decrypted_b == message)

print("\nЦИФРОВИЙ ПІДПИС АБОНЕНТА A")

signature_a = Sign(message, private_a)
signed_message_a = (message, signature_a)

print("S_A = M^d mod n =", signature_a)
print("Підписане повідомлення (M, S_A):")
print(signed_message_a)
print("Підпис A правильний:", Verify(*signed_message_a, public_a))

print("\nЦИФРОВИЙ ПІДПИС АБОНЕНТА B")

signature_b = Sign(message, private_b)
signed_message_b = (message, signature_b)

print("S_B = M^d1 mod n1 =", signature_b)
print("Підписане повідомлення (M, S_B):")
print(signed_message_b)
print("Підпис B правильний:", Verify(*signed_message_b, public_b))
```

```
print("\nПЕРЕВІРКА ЗМІНЕНОГО ПОВІДОМЛЕННЯ")

modified_message = message - 1

print("Змінене M =", modified_message)
print("Перевірка підпису A:", Verify(
    modified_message, signature_a, public_a
))
print("Перевірка підпису B:", Verify(
    modified_message, signature_b, public_b
))
```

Для демонстрації вибрано випадкове повідомлення M (діапазон $0 < M < \min(n, n_1)$), random.seed(7):

M =
28786320068520398613983744442528336046079289080110315272395293570801036655365245391556219491964904409131405259529767494775202716115523070271875110418975297

Шифрування для A:

$C_A = M^e \bmod n =$
38607243860739303652314633194211969349349150576274552569968703468945944355215135213620739036736370038166619243273192363718836803490463642080570723410880004

Розшифрування $C_A^d \bmod n$ повертає M — збігається.

Шифрування для B:

$C_B = M^{e_1} \bmod n_1 =$
72938457536841855127379238992385320676049050428726913678599375049541145385876192612343850211516727756389182657623277243905267079119114809514057224719655 3

Розшифрування $C_B^{d_1} \bmod n_1$ повертає M — збігається.

Цифровий підпис A:

$S_A = M^d \bmod n =$
93475219609164480782316963566086695439242158142149294383241830226938533125320476458482592655442215266487929040774479849064272508675550873750130378322992 01

Підписане повідомлення (M, S_A); перевірка підпису відкритим ключем A — правильний.

Цифровий підпис B:

$S_B = M^{d_1} \bmod n_1 =$
84983500982285439054272431226135567121090973476300693343829421513119971938204165081745366532625603532990436684099560335926614085503484388890873583590774 36

Підписане повідомлення (M, S_B); перевірка підпису відкритим ключем B — правильний.

Зміна повідомлення. Після заміни M на M − 1 перевірка і підпису A, і підпису B повертає false, тобто спотворення повідомлення виявляється.

Результат виконання part4.py (random.seed(7))

```
ВІДКРИТЕ ПОВІДОМЛЕННЯ
M =
28786320068520398613983744442528336046079289080110315272395293570801036655365245391556219491964904409131405259529767494775202716115523070271875110418975291
ШИФРУВАННЯ ДЛЯ АБОНЕНТА A
C_A = M^e mod n =
38607243860739303652314633194211969349349150576274552569968703468945944355215135213620739036736370038166619243273192363718836803490463642080570723410880041
M_A = C_A^d mod n =
28786320068520398613983744442528336046079289080110315272395293570801036655365245391556219491964904409131405259529767494775202716115523070271875110418975291
M_A == M: True

ШИФРУВАННЯ ДЛЯ АБОНЕНТА B
C_B = M^e1 mod n1 =
72938457536841855127379238992385320676049050428726913678599375049541145385876192612343850211516727756389182657623272439052670791191148095140572247196553
M_B = C_B^d1 mod n1 =
28786320068520398613983744442528336046079289080110315272395293570801036655365245391556219491964904409131405259529767494775202716115523070271875110418975291
M_B == M: True

ЦИФРОВИЙ ПІДПИС АБОНЕНТА A
S_A = M^d mod n =
93475219609164480782316963566086695439242158142149294383241830226938533125320476458482592655442215266487929040774479849064272508675550873750130378322992 01
Підписане повідомлення (M, S_A):
(28786320068520398613983744442528336046079289080110315272395293570801036655365245391556219491964904409131405259529767494775202716115523070271875110418975291,
93475219609164480782316963566086695439242158142149294383241830226938533125320476458482592655442215266487929040774479849064272508675550873750130378322992 01)
Підпис A правильний: True

ЦИФРОВИЙ ПІДПИС АБОНЕНТА B
S_B = M^d1 mod n1 =
84983500982285439054272431226135567121090973476300693343829421513119971938204165081745366532625603532990436684099560335926614085503484388890873583590774 36
Підписане повідомлення (M, S_B):
(28786320068520398613983744442528336046079289080110315272395293570801036655365245391556219491964904409131405259529767494775202716115523070271875110418975291,
84983500982285439054272431226135567121090973476300693343829421513119971938204
```

```
650817453665326256035329904366840995603359266140855034843888908735835907743 6)
Підпис B правильний: True

ПЕРЕВІРКА ЗМІНЕНОГО ПОВІДОМЛЕННЯ
Змінене M =
287863200685203986139837444425283360460792890801103152723952935708010366553652
453915562194919649044091314052595297674947752027161155230702718751104189752 8
Перевірка підпису A: False
Перевірка підпису B: False
```

8. Завдання 5. Протокол розсилання ключів A → B

Протокол реалізовано двома окремими процедурами: SendKey() для відправника та ReceiveKey() для отримувача. Нижче наведено код і результат одного виконання для випадкового ключа $0 < k < n$.

Програмний код (part5.py)

```
import random
from pathlib import Path

from part4 import load_keys, Encrypt, Decrypt, Sign, Verify


def SendKey(k, sender_private_key, receiver_public_key):
    _, p, q = sender_private_key
    n = p * q
    n1, _ = receiver_public_key

    if not 0 < k < n:
        raise ValueError("Ключ має задовольняти 0 < k < n")

    if n > n1:
        raise ValueError("Модуль відправника має бути не більшим за модуль отримувача")

    signature = Sign(k, sender_private_key)

    encrypted_key = Encrypt(k, receiver_public_key)
    encrypted_signature = Encrypt(signature, receiver_public_key)

    return encrypted_key, encrypted_signature


def ReceiveKey(encrypted_key, encrypted_signature,
               receiver_private_key, sender_public_key):
    _, p1, q1 = receiver_private_key
    n1 = p1 * q1
    n, _ = sender_public_key
```

```
    if n > n1:
        raise ValueError("Порушено умову n <= n1")

    k = Decrypt(encrypted_key, receiver_private_key)
    signature = Decrypt(encrypted_signature, receiver_private_key)

    if not 0 < k < n:
        raise ValueError("Отриманий ключ поза допустимим діапазоном")

    if not Verify(k, signature, sender_public_key):
        raise ValueError("Підпис неправильний — ключ відхилено")

    return k, signature


if __name__ == "__main__":
    filename = Path(__file__).resolve().with_name("keys.txt")
    public_a, private_a, public_b, private_b = load_keys(filename)

    n, e = public_a
    n1, e1 = public_b

    k = random.randrange(1, n)

    print("ПРОТОКОЛ ПЕРЕДАЧІ КЛЮЧА A -> B")
    print("\nПеревірка n <= n1:", n <= n1)

    print("\nВІДПРАВНИК A")

    print("1. Випадковий ключ:")
    print("k =", k)

    signature = Sign(k, private_a)

    print("\n2. Підпис ключа секретним ключем A:")
    print("S = k^d mod n =", signature)

    encrypted_key, encrypted_signature = SendKey(
        k, private_a, public_b
    )

    print("\n3. Шифрування відкритим ключем B:")
    print("k1 = k^e1 mod n1 =", encrypted_key)
    print("S1 = S^e1 mod n1 =", encrypted_signature)
```

```
print("\n4. Відкритим каналом передаємо пару (k1, S1):")
print((encrypted_key, encrypted_signature))

print("\nОТРИМУВАЧ B")

received_key, received_signature = ReceiveKey(
    encrypted_key,
    encrypted_signature,
    private_b,
    public_a,
)

print("\n5. Розшифрування секретним ключем B:")
print("k = k1^d1 mod n1 =", received_key)
print("S = S1^d1 mod n1 =", received_signature)

recovered_value = Encrypt(received_signature, public_a)

print("\n6. Перевірка підпису відкритим ключем A:")
print("S^e mod n =", recovered_value)
print("Підпис правильний:", Verify(
    received_key, received_signature, public_a
))

print("\n7. Результат:")
print("Отриманий ключ збігається з початковим:", received_key == k)

print("\nПЕРЕВІРКА СПОТВОРЕНОГО ПАКЕТА")

modified_signature = (encrypted_signature + 1) % n1

try:
    ReceiveKey(
        encrypted_key,
        modified_signature,
        private_b,
        public_a,
    )
    print("Ключ прийнято")
except ValueError as error:
    print("Ключ відхилено:", error)

print("\nКонфіденційність:")
print("Шифрування відкритим ключем B, розшифрування секретним ключем B.")
```

```
    print("\nПідтвердження справжності відправника:")
    print("Підпис секретним ключем A, перевірка відкритим ключем A.")
```

Виконання протоколу для випадкового ключа k, 0 < k < n (random.seed(11)):

Крок 1. Відправник A обирає ключ:

k =
637894204862233192671333323089098289600525042213533093770052624178053163183306
523671333588314350496383621103784795116447260058697233215235089499692093579O

Крок 2. A підписує ключ власним секретним ключем:

$S = k^d \bmod n =$
765537757217905572145028685304340384735042604194988826220625078437911170336921
226635943302636742465780863545102004047150452501039917563921623580618519383 2

Крок 3. A шифрує k і S відкритим ключем B:

$k_1 = k^{e_1} \bmod n_1 =$
184341247277712216728609920564840169732539840219680608062931363559080714301428
782256296316059526102083274912560652927829754695813302461139703111414493505 9

$S_1 = S^{e_1} \bmod n_1 =$
980467076458103284361862770646027408290068234145364388565458100898768362593178
006314633440527643944332213980057343618450697066737945121142291681031893289 3

Крок 4. По відкритому каналу передається пара (k_1, S_1).

Крок 5. B розшифровує секретним ключем d_1: $k = k_1^{d_1} \bmod n_1$, $S = S_1^{d_1} \bmod n_1$; отримане k збігається з початковим.

Крок 6. B перевіряє підпис відкритим ключем A: $S^e \bmod n = k$ — рівність виконується, підпис правильний.

Результат: отриманий ключ збігається з надісланим.

Спотворений пакет. Якщо підпис у переданій парі змінити ($S_1 + 1 \bmod n_1$), після розшифрування перевірка підпису не виконується, і ReceiveKey відхиляє ключ повідомленням «Підпис неправильний — ключ відхилено».

Яким ключем що забезпечується:

- **конфіденційність** — шифрування відкритим ключем B (e_1, n_1) і розшифрування лише секретним ключем d_1 абонента B: стороння особа не може отримати k;
- **підтвердження справжності відправника** — підпис секретним ключем d абонента A і перевірка відкритим ключем (e, n) абонента A: підписати міг лише власник d.

Результат виконання part5.py (random.seed(11))

ПРОТОКОЛ ПЕРЕДАЧІ КЛЮЧА A -> B

Перевірка n <= n1: True

ВІДПРАВНИК A
1. Випадковий ключ:
k =
637894204862233192671333323089098289600525042213533093770052624178053163183306
523671333588314350496383621103784795116447260058697233215235089499692093579O

2. Підпис ключа секретним ключем A:
S = k^d mod n =
765537757217905572145028685304340384735042604194988826220625078437911170336921
226635943302636742465780863545102004047150452501039917563921623580618519383 2

3. Шифрування відкритим ключем B:
k1 = k^e1 mod n1 =
184341247277712216728609920564840169732539840219680608062931363559080714301428
782256296316059526102083274912560652927829754695813302461139703111414493505 9
S1 = S^e1 mod n1 =
980467076458103284361862770646027408290068234145364388565458100898768362593178
006314633440527643944332213980057343618450697066737945121142291681031893289 3

4. Відкритим каналом передаємо пару (k1, S1):
(184341247277712216728609920564840169732539840219680608062931363559080714301428
782256296316059526102083274912560652927829754695813302461139703111414493505 9,
980467076458103284361862770646027408290068234145364388565458100898768362593178
006314633440527643944332213980057343618450697066737945121142291681031893289 3)

ОТРИМУВАЧ B

5. Розшифрування секретним ключем B:
k = k1^d1 mod n1 =
637894204862233192671333323089098289600525042213533093770052624178053163183306
523671333588314350496383621103784795116447260058697233215235089499692093579O
S = S1^d1 mod n1 =
765537757217905572145028685304340384735042604194988826220625078437911170336921
226635943302636742465780863545102004047150452501039917563921623580618519383 2

6. Перевірка підпису відкритим ключем A:
S^e mod n =
637894204862233192671333323089098289600525042213533093770052624178053163183306
523671333588314350496383621103784795116447260058697233215235089499692093579O
Підпис правильний: True

7. Результат:
Отриманий ключ збігається з початковим: True

ПЕРЕВІРКА СПОТВОРЕНОГО ПАКЕТА
Ключ відхилено: Підпис неправильний — ключ відхилено

Конфіденційність:
Шифрування відкритим ключем B, розшифрування секретним ключем B.

Підтвердження справжності відправника:
Підпис секретним ключем A, перевірка відкритим ключем A.

9. Перевірка взаємодії із тестовим середовищем

Усі операції перевірені в обидві сторони за допомогою сервісу asymcryptwebservice.appspot.com. Значення на сайті вводяться в режимі Bytes (hex), тому повідомлення 12345 відповідає 3039 (hex). Модуль нашого ключа абонента A у шістнадцятковому вигляді на скриншотах починається з 52BF2669…, показник e = 10001 (hex) = 65537.

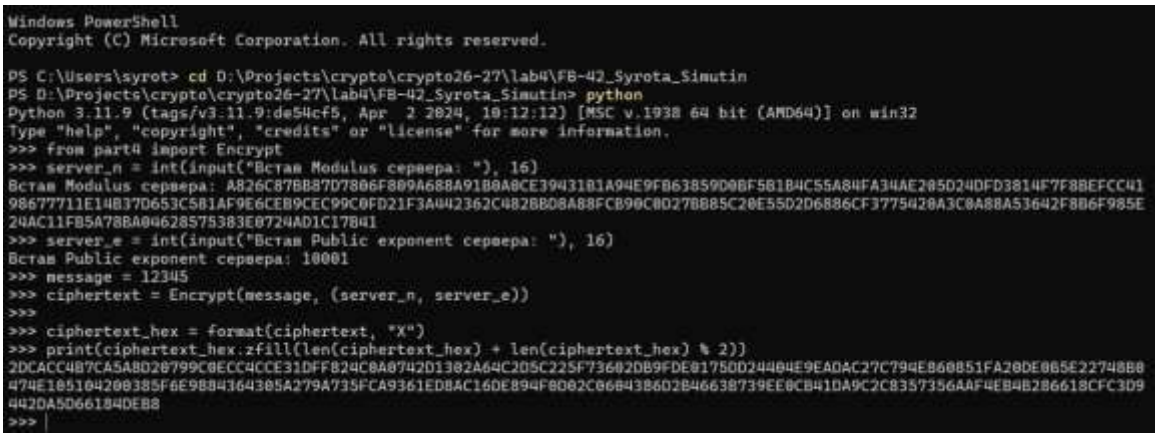
№	Перевірка	Результат
1	Encrypt (наш код) → Decrypt (сервер): повідомлення 12345 зашифроване відкритим ключем сервера (1024 біти)	Сервер розшифрував: 3039 (hex) = 12345 — збігається
2	Encrypt (сервер) → Decrypt (наш код): повідомлення 3039 зашифроване на сервері для нашого ключа	Наш Decrypt повернув 12345 — збігається
3	Sign (наш код) → Verify (сервер): підпис повідомлення 12345	Verification: true
4	Sign (сервер) → Verify (наш код): підпис повідомлення 3039	Verify повернув True
5	SendKey (наш код) → Receive key (сервер): k = 12345, відкритий ключ сервера 1024 біти	Key: 3039, Verification: true
6	Send key (сервер) → ReceiveKey (наш код): серверний ключ 256 біт (n_сервера ≤ n_A)	Отримано ключ 8924779698611494162 (hex 7BDB2FD9B25B1D12), підпис сервера правильний: True

9.1. Отримання ключа сервера та шифрування

Спочатку з сайту отримано відкритий ключ сервера розміром 1024 біти (модуль починається з A826C87B…, показник 10001 у hex = 65537). Його довжина більша за наш модуль (512 біт), тому він підходить для передачі підписаного ключа від A до сервера.



Далі наша функція Encrypt() зашифрувала повідомлення M = 12345 відкритим ключем сервера. Результат виведено в hex для вставки на сайт.



Криптограму вставлено в розділ Decryption на сайті. Сервер повернув 3039 (hex), тобто 12345 — розшифрування сервером збігається з початковим повідомленням.



У зворотному напрямку виведено наш модуль і показник у hex, щоб підставити їх на сайт.

```
>>> from pathlib import Path
>>> from part4 import load_keys, Decrypt
>>>
>>> public_a, private_a, public_b, private_b = load_keys(Path("keys.txt"))
>>>
>>> print("Modulus:", format(public_a[0], "X"))
Modulus: 52BF2669202D67A842422F4FDBD534DA28037D6B45766B4F08780E759B595666245D7D30D8C7909525C68E264D16F3728835273248B70A08
9A9EF8E10B12567F5
>>> print("Public exponent:", format(public_a[1], "X"))
Public exponent: 10001
>>>
```

Сервер зашифрував повідомлення 3039 (hex) нашим відкритим ключем.



Отриману криптограму вставлено в нашу функцію Decrypt(), що використовує секретний ключ A. Розшифроване значення 12345 збігається з початковим (результат True).

```
>>> server_ciphertext = int(input("Встав Ciphertext із сайту: "), 16)
Встав Ciphertext із сайту: 100EF5DDD2101CD84FE8643FB44187CBC192D7FB0132B2CC0D526F3017C9FBD80C75DFD6776708CAB163BD275FAC5
E2EFC414741802AD263C4CEC6289ED83664
>>> decrypted = Decrypt(server_ciphertext, private_a)
>>>
>>> print("Розшифроване повідомлення:", decrypted)
Розшифроване повідомлення: 12345
>>> print("Збігається з початковим:", decrypted == 12345)
Збігається з початковим: True
>>>
```

Отже, шифрування і розшифрування сумісні із сервером в обох напрямках.

9.2. Цифровий підпис

Наша функція Sign() підписала повідомлення 12345 секретним ключем A; підпис, модуль і показник виведено в hex.

```
>>> from part4 import Sign
>>>
>>> message = 12345
>>> signature_a = Sign(message, private_a)
>>>
>>> signature_hex = format(signature_a, "X")
>>> print("Signature:", signature_hex.zfill(len(signature_hex) + len(signature_hex) % 2))
Signature: 342D094E3DA5E648DA1195F839C93D0D389D7D43837C788E9F30D181C0C9FDC48B441007B90C878F19879D6B4013A996961FBFEF36EAA
504D3FE7CA8D0CEB195
>>> print("Modulus:", format(public_a[0], "X"))
Modulus: 52BF2669202D67A842422F4FDBD534DA28037D6B457668B4F08780E759B5956662A5D7D30D8C7909525C68E264D16F372883527324B70A08
9A9EF0E10B12567F5
>>> print("Public exponent:", format(public_a[1], "X"))
Public exponent: 10001
>>>
```

На сайті в розділі Verify введено повідомлення 3039, наш підпис, модуль і e. Сервер підтвердив підпис (Verification: true).



У зворотному напрямку сервер підписав повідомлення 3039 своїм секретним ключем.



Наша функція Verify() перевірила цей підпис відкритим ключем сервера і повернула True.

```
>>> server_signature = int(input("Встав Signature із сайту: "), 16)
Встав Signature із сайту: 9242F10C09B623D408AA9EAF07DB1FA8F1B4E612585DCB735DCB4336D636ACBD1B81EC66624876A82F7E15B7699BE5
9E27D38949CA38C1156194445E11BB4DD4A30DCE5599E2D902C05563BEB9EEE287C08FBD1F8E54258B4E91D8C0F5AAA399652CEB565C185D45CF10FB
2BA9468D9A469A5446CD757022C2FBE0984BAE4C28
>>> print("Підпис сервера правильний:", Verify(
...     12345, server_signature, (server_n, server_e)
... ))
Підпис сервера правильний: True
>>>
```

Отже, підписи сумісні із сервером в обох напрямках.

9.3. Протокол передачі ключа: A → сервер

Функція SendKey() сформувала для k = 12345 пару (Key, Signature) для відкритого ключа сервера (1024 біти). Значення Key збігається з криптограмою з рисунка 2, бо k і ключ сервера ті самі, а шифрування без доповнення детерміноване.

```
>>> from part5 import SendKey
>>>
>>> k = 12345
>>> k1, s1 = SendKey(k, private_a, (server_n, server_e))
>>>
>>> k1_hex = format(k1, "X")
>>> s1_hex = format(s1, "X")
>>>
>>> print("Key:", k1_hex.zfill(len(k1_hex) + len(k1_hex) % 2))
Key: 2DCACC4B7CA5A8D20799C0ECC4CCE31DFF824C0A0742D1302A64C2D5C225F73602DB9FDE0175DD24404E9EADAC27C794E860851FA20DE0B5E22
748B047AE10510420038 5F6E9884364305A279A735FCA9361ED8AC16DE894F8D02C0604386D2B4663B739EE0CB41DA9C2CB357356AAF4EB4B286618C
FC3D9442DA5D66184DEB8
>>> print("Signature:", s1_hex.zfill(len(s1_hex) + len(s1_hex) % 2))
Signature: 7FB03C2DB936AC18BCA6FD19B87369E8CD632B991881E1608B93E2DE1261BC133366AE08DA1652838CDD29AC2A4EBCBCB95ACBA26411C
31A057FFD930792B58A6C9FFE7766BBF248750D8EEE560B691F64DB3E3D8CA942E33A831D11C35172ED6855B2F5AD53809726 0CBA892356096F490B6
5129BDFB598FABE58DC4CBD94AA
>>> print("Modulus:", format(public_a[0], "X"))
Modulus: 52BF2669202D67A842422F4FDBD534DA28037D6B45766B4F08780E759B595666245D7D30D8C7909525C68E264D16F372883527324B70A08
9A9EF8E10B12567F5
>>> print("Public exponent:", format(public_a[1], "X"))
Public exponent: 10001
>>>
```

Пару вставлено в розділ Receive key на сайті разом з нашим модулем і e. Сервер розшифрував ключ (3039 = 12345) і підтвердив підпис відправника (Verification: true).



9.4. Протокол передачі ключа: сервер → A

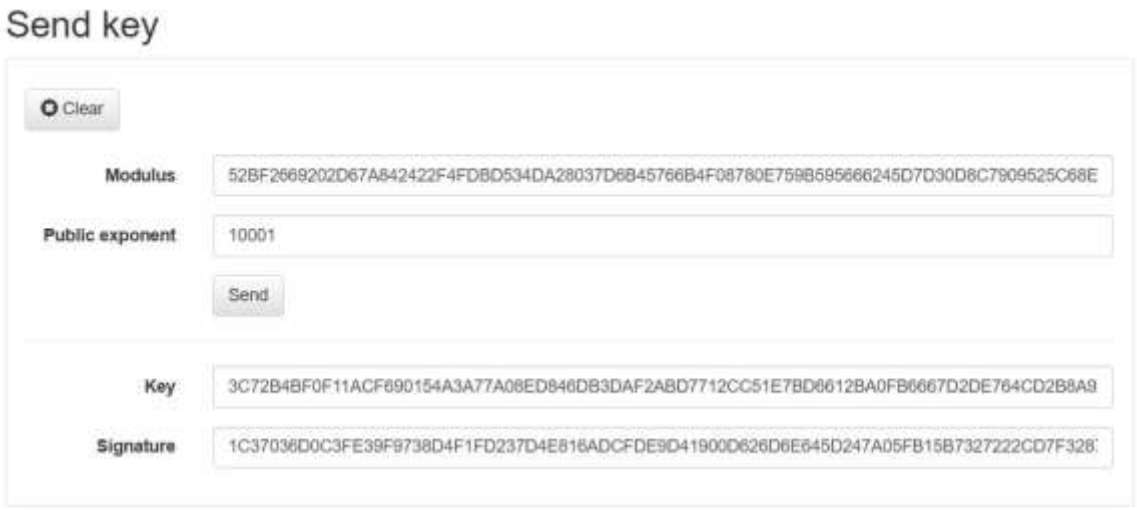
Умова протоколу вимагає, щоб модуль відправника не перевищував модуль отримувача. Тому для напрямку сервер → A запрошено ключ сервера на 256 біт (модуль B0B3245D…).



Наш код перевірив порядок модулів: server_n ≤ n, тобто True.

```
>>> server_n = int(input("Встав новий Modulus сервера: "), 16)
Встав новий Modulus сервера: B0B3245D4BD9F23A5A73F7589C2853F8D6FCDA9BC97AC4A1569E7B391B0F54EB
>>> server_e = int(input("Встав новий Public exponent сервера: "), 16)
Встав новий Public exponent сервера: 10001
>>> print("Порядок модулів правильний:", server_n <= public_a[0])
Порядок модулів правильний: True
>>> print("Наш Modulus:", format(public_a[0], "X"))
Наш Modulus: 52BF2669282D67A842422F4FDBD534DA28837D6B45766B4F08788E759B595666245D7D38D8C7909525C68E264D16F372883527324B7
8AB89A9EF8E10B12567F5
>>> print("Наш Public exponent:", format(public_a[1], "X"))
Наш Public exponent: 10001
>>>
```

На сайті в розділі Send key введено наш модуль і e; сервер сформував пару Key і Signature для нашого ключа.



Пару вставлено в нашу функцію ReceiveKey(): вона розшифрувала k і S секретним ключем A та перевірила підпис відкритим ключем сервера. Отримано ключ 8924779698611494162 (hex 7BDB2FD9B25B1D12), підпис сервера правильний (True).

```
>>> from part5 import ReceiveKey
>>>
>>> incoming_key = int(input("Встав Key із сайту: "), 16)
Встав Key із сайту: 3C72B4BF0F11ACF690154A3A77A08ED846DB3DAF2ABD7712CC51E7BD66128A0FB6667D20E764CD2B8A929342CFEB6582DC95
03C43BA70EEC8D0278B7C7F16464
>>> incoming_signature = int(input("Встав Signature із сайту: "), 16)
Встав Signature із сайту: 1C37B36D0C3FE39F9738D4F1FD237D4E816ADCFDE9D41900D626D6E6450247A05F815B7327222CD7F3287CDC48E47E
E57DE8C911C3932DD1E7F553126CD5F2F9
>>> received_key, received_signature = ReceiveKey(
...     incoming_key,
...     incoming_signature,
...     private_a,
...     (server_n, server_e),
... )
>>>
>>> print("Отриманий ключ:", received_key)
Отриманий ключ: 8924779698611494162
>>> print("Отриманий ключ HEX:", format(received_key, "X"))
Отриманий ключ HEX: 7BDB2FD98258 1D12
>>> print("Підпис сервера правильний:", Verify(
...     received_key, received_signature, (server_n, server_e)
... ))
Підпис сервера правильний: True
>>>
```

Це підтверджує, що SendKey() і ReceiveKey() сумісні із сервером в обох напрямках.

10. Відповіді на контрольні питання

1. Які перевірки виконуються програмою під час генерації простих чисел і для чого потрібна кожна з них?

Формується непарне число потрібної довжини (старший біт гарантує розмір, молодший — непарність). Пробні ділення на малі прості числа швидко і дешево відкидають значну частку складених кандидатів до виконання дорогих обчислень. Тест Міллера—Рабіна з 40 основами визначає складеність: для складеного числа кожна випадкова основа виявляє його складеність з імовірністю щонайменше 3/4, тому ймовірність помилково прийняти складене число не перевищує 4^{-40}. Також перевіряється різність $p \neq q$.

2. Які умови повинні виконуватися для параметрів згенерованої пари ключів RSA? Як їх було перевірено у вашій реалізації?

Умови: p і q прості та різні; $n = pq$; $2 \leq e \leq \varphi(n) - 1$ та $\gcd(e, \varphi(n)) = 1$; $ed \equiv 1 \pmod{\varphi(n)}$; для обміну ключами $n \leq n_1$. У реалізації: $p \neq q$ перевіряється в GenerateKeyPair, $\gcd(e, \varphi(n)) = 1$ — перед обчисленням d, d знаходиться розширеним алгоритмом Евкліда, а у виводі програми напечатано результати перевірок $n = pq$, діапазону e, gcd та $e \cdot d \bmod \varphi(n) = 1$; порядок модулів $n \leq n_1$ гарантується в generate_two_pairs та перевіряється у виводі.

3. Які дані необхідні для виконання операцій Encrypt(), Decrypt(), Sign() та Verify()? Чому кожній із цих операцій не потрібні всі параметри RSA?

Encrypt і Verify потребують лише відкритий ключ (n, e) (плюс повідомлення, а для Verify — ще й підпис): ці операції може виконувати будь-хто. Decrypt і Sign потребують лише секретні дані (d та модуль $n = pq$) і відповідне повідомлення або криптограму. Параметри, що не потрібні операції, не повинні передаватись: так зменшується ризик витоку секретних значень (наприклад, $\varphi(n)$, d, p, q), а відкрита частина не виявляє секрету.

4. Які властивості забезпечуються на різних етапах реалізованого протоколу передачі ключа: конфіденційність та підтвердження справжності відправника?

Підпис $S = k^d \bmod n$, створений секретним ключем A, забезпечує підтвердження справжності відправника: його перевіряє B відкритим ключем A. Шифрування пари (k, S) відкритим ключем B ($k_1 = k^{e_1}$, $S_1 = S^{e_1} \bmod n_1$) забезпечує конфіденційність: розшифрувати їх може лише B за допомогою d_1. Підписане значення S також шифрується, щоб підпис не міг бути використаний стороннім спостерігачем для перевірки здогадок щодо k.

5. Які перевірки повинен виконати одержувач у реалізованому протоколі передачі ключа, перш ніж вважати отриманий ключ коректним і справжнім?

Одержувач перевіряє: (1) виконання умови $n \le n_1$ для модулів відправника та отримувача; (2) що k_1 та S_1 лежать у діапазоні $[0, n_1)$; (3) що розшифроване k лежить у діапазоні $0 < k <$ n відправника; (4) що підпис правильний — $S^e \bmod n = k$ за відкритим ключем відправника. Якщо хоча б одна перевірка не виконується, ключ відхиляється (у реалізації — виняток `ValueError`).

11. Висновки

У роботі власноруч реалізовано всі необхідні примітиви криптосистеми RSA: швидке піднесення до степеня за модулем за схемою Горнера, тест Міллера—Рабіна з пробними діленнями для генерації 256-бітних простих чисел, розширений алгоритм Евкліда для обчислення секретного показника, а також процедури GenerateKeyPair, Encrypt, Decrypt, Sign, Verify, SendKey та ReceiveKey.

Згенеровано дві пари простих чисел p, q та p_1, q_1 довжиною 256 біт та побудовано 512-бітні модулі $n \le n_1$. Перевірено співвідношення, необхідні для коректності RSA ($p \ne q$, $n = pq$, $\gcd(e, \varphi(n)) = 1$, $ed \equiv 1 \pmod{\varphi(n)}$). Показано, що шифрування і розшифрування, підпис і перевірка працюють коректно, а зміна повідомлення чи підпису виявляється. Протокол передачі ключа з підтвердженням справжності забезпечує конфіденційність (шифрування відкритим ключем отримувача) та автентифікацію (підпис секретним ключем відправника), а умова $n \le n_1$ необхідна для коректної передачі підпису.

Усі операції реалізації також успішно перевірені у взаємодії із зовнішнім тестовим сервісом в обох напрямках, що підтверджує сумісність реалізації зі стандартною схемою RSA.